

PRAHARI — an early-warning system for India's oil supply

PRAHARI watches the world for trouble that could choke off India's oil supply. The moment something looks dangerous, it does the work a room full of analysts would normally take weeks to do: it figures out how much the trouble costs India (in rupees, and in days of supply left), and it tells the people who buy crude what to buy instead and how to ship it. "Prahari" means watchman.

Team f20230436. Problem statement **PS 2: AI-Driven Energy Supply Chain Resilience for Import-Dependent Economies.**

Our one-line pitch: **47 days versus 47 seconds.**

Why we picked this problem

India buys about 88% of its crude oil from other countries. Close to 40% of that oil passes through one narrow gap of water next to Iran, the Strait of Hormuz. If that strait becomes dangerous, a huge share of the country's oil is suddenly in doubt.

The trouble is speed. When a crisis hits, prices jump, refiners start scrambling, and it takes weeks to line up other supply. McKinsey measured it: a country that isn't prepared loses roughly 47 extra days getting back to normal after an oil shock. India's emergency reserve only holds about 9 and a half days of cover.

Here is the gap we went after, in plain terms. The danger shows up in the news and in ship movements almost the instant it happens. The response takes weeks. We wanted to shrink that from weeks to seconds.

What it actually does

Think of an ordinary day. Oil prices tick along, ships move through Hormuz, nothing unusual, and our dashboard shows a calm map with a low risk score.

Now a threat appears. Say news breaks of a strike near the strait. Everything below runs in a single pass:

1. **It reads the risk.** It gathers the recent news events for that sea corridor, weighs each one by how serious and how recent it is, and boils that down to a single risk number between 0 and 1.
2. **It simulates the damage.** It runs 10,000 fast simulations of where the oil price could head, then follows that through to what it means for India: how much bigger the daily import bill gets, and how quickly the emergency reserve drains.
3. **It finds the way out.** Using a map of who supplies India's crude and the routes they use, it works out which suppliers can still deliver while steering clear of the dangerous chokepoints, ranks them by speed and safety, and draws the new route on screen.
4. **It writes the answer in one line.** Something a decision-maker reads in five seconds: the risk level, the cost per day, the reserve days left, and the supplier and route to switch to.

From the signal arriving to the recommendation appearing, this runs in about 20 milliseconds on our laptop. The "47 seconds" is the promise on the slide; the real number is much faster than that.

A real run from our data: the Strait of Hormuz reads Critical at 0.90, backed by 77 actual news events, with an estimated cost of roughly Rs 1,300 crore a day, and the recommended fix is to reroute through the UAE's Fujairah port, about 3 days out. Nobody typed that answer in. It falls out of the data.

The idea that makes it fast

Most price-shock models have a hard job built in: they have to guess *when* a shock will strike, because they only ever see it after prices have already moved. We sidestep that. We watch the news and the ships directly, so we catch the shock while it is forming. We never have to guess the timing. We only have to size the shock and estimate how long it lasts. That is a smaller and more honest job, and it is the reason the whole system can answer in near real time.

What's under the hood

Six data feeds. Oil prices from the US government (EIA) and from Yahoo Finance, the US sanctions list (OFAC), live ship positions (AIS), India's monthly import figures (PPAC), and world news events (GDELT). Small scripts pull each one and drop it into one shared database.

One database. Everything lands in a single cloud Postgres database, so the map, the model, and the agents all read the exact same numbers. No two parts of the system can disagree about what the data says.

A knowledge graph. A small map of India's real crude supply chain: 7 suppliers, their export ports, 5 sea chokepoints, Indian ports, and 5 refineries, wired together the way the real infrastructure is (including the pipelines that bypass Hormuz, like the UAE's line to Fujairah). The map itself is fixed, but the danger on each chokepoint updates from the live news. Ask it "who can still reach Jamnagar today" and it answers with real routes and travel times.

A scenario model. A price-shock simulation (jump-diffusion, 10,000 runs) that turns a risk score into a spread of outcomes: the likely price jump, the worst-case jump, how far the reserve falls, and the cost per day. It reports a range, not a single fake-precise number.

An agent pipeline. The four steps above are wired into one chain using LangGraph, so a detected signal actually drives the whole thing end to end and we can show each step as it runs.

A dashboard. A single-screen React and Leaflet console: a live map, a risk dial, the chokepoint watch list, the simulation chart, the cost in rupees and days, and the ranked list of suppliers to switch to. There is also a panel of assumptions you can change on the spot and watch everything recompute.

The one rule we stuck to: the maths is code, only the words come from AI

Every number in the system is computed and can be checked by hand: the risk score, the simulation, the travel times, the cost. The only place we use a language model is to read messy news text and turn it into clean events, and (in one version) to write the summary sentence. No

figure is ever invented by an AI. That keeps the whole thing auditable, which matters when you are telling a government what a crisis costs.

Is the data real?

Yes. All of it is real, public data. Real news, the real US sanctions list, real oil prices, real Indian import figures, real ship positions. We made none of it up.

We should be straight about one thing, though: it is not a live stream. Each feed refreshes on its own clock. Prices and ship positions update every ten minutes or so, news every few hours, and the slower government figures daily or monthly. For the demo we run on a recent real snapshot and refresh it just before we present. One honest limit: the free ship-tracking service barely covers the Persian Gulf, so the live ship dots you see sit on well-covered lanes like the English Channel, there to prove the pipeline really is live. The Hormuz story is told with the data we do have, which is the geography, the sanctioned tankers, and the news signal.

Our rule the whole way through: real but slightly old beats made-up. We never invent a number to fill a gap.

What we kept simple on purpose

Time was short, so we cut things that would have added risk on stage without changing the story. The final summary sentence comes from a template rather than an AI, so nothing can hang at the worst possible moment. We run one price model instead of two. We left out the heavier real-time streaming setup and a couple of other extras. Every one of those was a choice, not an oversight, and we are happy to walk a judge through the reasoning on any of them.

How to run it

See [DEMO_GUIDE.md](#) for the step-by-step. The very short version:

```
./run_dev.sh
```

then open <http://localhost:5173> and click **Inject signal**. That button runs the whole pipeline on a cached real event with no database and no internet, so it works even if the wifi dies.

The team

Team **f20230436**

- Ansh Sharma (team lead)
- Harsh Vardhan
- Aditya Nitin Jagtap
- Abhijay Pansari

The four of us split the build across four areas: the data layer (six live feeds into one shared database), the news pipeline that turns raw world events into scored risk signals, the agents and the knowledge graph, and the backend API and the dashboard.

PRAHARI — how it's built

Team f20230436 · PS 2: AI-Driven Energy Supply Chain Resilience for Import-Dependent Economies

This is the technical companion to the project overview. It walks through how the system is put together, the calls we made along the way, and the numbers to back them up. If you only read one thing, read the two ideas in the next two sections. They are what the whole project rests on.

Idea 1: we watch the shock form instead of guessing when it arrives

A normal oil price-shock model treats the shock as a random event on a hidden clock. It has to estimate *when* a jump might happen and how often, and it can only do that after prices have already moved. That is the hard, uncertain part.

We removed that part. PRAHARI reads the news and the ship movements directly, so it sees the disruption while it is still forming. It never has to guess the timing. All it has to do is measure how large the shock is and how long it takes to settle. That is a smaller and more honest problem, and it is the reason the system can go from a detected signal to a full recommendation in milliseconds rather than after the fact.

Idea 2: every number is computed, only the words come from an AI

We drew a hard line through the system. Everything quantitative, which is the risk score, the simulated price paths, the reserve-cover days, the travel times, the cost per day, is computed with plain code (SQL, numpy, and a graph). A language model is used in exactly two narrow places: to read messy news text and turn it into clean structured events, and, optionally, to write the final summary sentence.

This one rule buys three things:

- **You can audit it.** Every figure on screen can be recomputed by hand and traced back to the events that produced it. When you tell a government what a crisis costs, that matters.
- **It does not fall over.** The critical path has no live AI call in it, so there is nothing to hang or rate-limit at the worst moment. The summary line is a template. If the news-reading model is down, extraction falls back to rules and the pipeline still finishes.
- **It is fast.** No network round-trip to an AI sits between the signal and the answer.

The shape of the system

six public feeds (EIA, yfinance, OFAC, AIS, PPAC, GDELT)	-> loaders (one small script per source)	-> shared Postgres (single source of truth)	-> read API (FastAPI)	-> dashboard (React + Leaflet)
				scenario model (Monte Carlo)
				knowledge graph (live risk)
				agent pipeline (LangGraph)

Nothing talks to the outside websites except the loaders. Everything else, the map, the model, the agents, reads from the same database, so no two parts can disagree about what the data says. That single design choice removed a whole class of "your JSON does not match mine" bugs between four people working in parallel.

The data layer

Six feeds, each with its own small loader that pulls fresh data, cleans it, and upserts it into one table. All loaders are idempotent, so re-running anything is always safe. What is loaded right now:

- Daily Brent and WTI spot prices from the US government (EIA): 500 rows
- Intraday price ticks from Yahoo Finance: ~14,700 rows
- The US sanctions list (OFAC SDN): ~19,100 entries, ~1,500 of them vessels
- Live ship positions (AISStream): ~1,550 positions per snapshot
- India's monthly crude import bill (PPAC): 312 rows
- World news events scored into risk signals (GDELT): 480 events

A single controller reruns each feed on the cadence its source actually changes (prices every ~10 minutes, news every few hours, sanctions daily, imports monthly) and prunes only stale ship positions. It keeps price, import, and event history forever, because the scenario model calibrates on that history and the risk score already down-weights old news by age.

Two honest notes we put in front of judges rather than hide: the free ship-tracker has almost no coverage near the Persian Gulf, so the live ship dots sit on well-covered lanes to prove the pipeline is real; and the data refreshes on a schedule, it is not a live stream. Our rule throughout: real but slightly old beats made-up.

Turning news into a risk score

For a sea corridor, the risk score is a confidence- and recency-weighted average of event severity:

```
risk = sum( severity/10 * confidence * decay ) / sum( confidence * decay )
decay = 0.5 ^ ( days_old / 7 )
```

An event from a week ago counts half as much as today's. It is a transparent heuristic, not a black box, which means we can explain any number on stage. When a score is backed by only a handful of events, the API flags it as low-evidence rather than pretending a 0.9 from three events is as solid as a 0.9 from three hundred. Corridor assignment is geofenced by the event's real coordinates, so a strike *at* Hormuz and a protest inland in Iran are no longer treated as the same thing.

The knowledge graph

A directed graph of India's real crude supply chain: 7 suppliers, their export ports, 5 sea chokepoints, Indian ports, and 5 refineries, wired the way the real infrastructure is, including the pipelines that bypass Hormuz (Saudi Arabia's line to Yanbu, the UAE's to Fujairah, Russia's to the Pacific). Iraq and Kuwait have no bypass, and the graph reproduces that true fact.

The skeleton is fixed, but the danger on each chokepoint is pulled live from the same news score at query time. So asking the graph "who can still reach Jamnagar today, by which route, in how many days" is a real path query over live risk, not a hardcoded answer. In one live run with Hormuz at 0.90 and the Red Sea also high, the graph returned: UAE via Fujairah in about 3 days, Nigeria via the Cape in ~26 days, the USA via the Cape in ~34 days, and correctly declared Iraq and Kuwait stuck. Nobody wrote that table. It fell out of the graph.

The scenario model

A mean-reverting jump-diffusion for Brent, run as 10,000 vectorised numpy paths in a few milliseconds. The jump is applied once, at the moment the signal fires, scaled by the corridor's risk score, and then decays back over a "days to reroute" window. From the price paths we derive a spread of outcomes: the likely and worst-case price jump, how far the reserve cover falls, the probability it drops below a safe floor, and the cost per day in rupees. We report ranges, not one fake-precise number, and we label the price-to-economy step honestly as a simplified elasticity rather than dressing it up as a full macro model.

The agent pipeline

The four steps (score the signal, simulate the scenario, rank the routes, write the summary) are wired into one linear chain with LangGraph. It runs synchronously and records each step's real wall-clock time, which is what the dashboard's latency read-out shows. There is an "inject signal" mode that runs the entire pipeline from a single cached real event and never touches the database, so the demo works even with no network. That is our on-stage safety net, and it is a real property of the code, not a mock.

Engineering we stand behind

- 36 automated tests plus a couple of integration suites, run on every push through GitHub Actions.
- Loaders validate their inputs: crude prices outside \$1 to \$500 a barrel are rejected as errors, and a truncated sanctions download fails loudly instead of quietly loading half a list.
- Downloads retry with backoff so a single network blip in the cloud refresher does not drop a feed.
- The read API degrades gracefully against an older database that predates a column, rather than crashing.
- Measured latency from signal to recommendation: about 20 milliseconds in our runs.
- Everything runs on free tiers. No paid infrastructure.

What we cut on purpose, and why

Being deliberate about scope is a feature, not an apology. These were choices:

We cut	Why
AI-generated summary on the critical path	A template has zero latency and zero failure risk right before the punchline. A judge cannot tell them apart.

Two price archetypes	One calibrated, severity-scaled jump does the same job with half the code and half the surface to defend live.
Real-time streaming (SSE, Redis)	A scripted reveal using the real recorded step timings gives the same visual effect without the streaming infrastructure risk.
Docker and NGINX	We present from a laptop. Containerising a single demo run buys nothing.
A separate reserve agent and a CVaR optimiser	Stretch goals. We said them in the pitch instead of half-building them.

Tech stack

Python and FastAPI for the service, Postgres on Neon for the shared database, numpy for the Monte Carlo, networkx for the knowledge graph, LangGraph for the agent chain, React with Vite and Leaflet for the dashboard, GitHub Actions for CI and the scheduled data refresh. A language model (via an OpenAI-compatible API) reads the news; nothing else in the system depends on it.